

Article

Federated Multi-Agent DRL for Task Offloading in Vehicular Edge Computing

Hongwei Zhao, Yu Li ^{*}, Zhixi Pang and Zihan Ma 

School of Intelligent Science and Information Engineering, Shenyang University, Shenyang 110003, China; zhw30@163.com (H.Z.); dbzxxz89@163.com (Z.P.); zihan022@126.com (Z.M.)

* Correspondence: liwenyu0229@163.com

Abstract

With the expansion of vehicle-to-everything (V2X) networks and the rising demand for intelligent services, vehicle edge computing encounters heightened requirements for more efficient task offloading. This study proposes a task offloading technique that utilizes federated collaboration and multi-agent deep reinforcement learning to reduce system latency and energy consumption. The task offloading issue is formulated as a Markov decision process (MDP), and a framework utilizing the Multi-Agent Dueling Double Deep Q-Network (MAD3QN) is developed to facilitate agents in making optimal offloading decisions inside intricate environments. Secondly, Federated Learning (FL) is implemented during the training phase, leveraging local training outcomes from many vehicles to enhance the global model, thus augmenting the learning efficiency of the agents. Experimental results indicate that, compared to conventional baseline algorithms, the proposed method decreases latency and energy consumption by at least 10% and 9%, respectively, while enhancing the average reward by at least 21%.

Keywords: vehicle edge computing; task offloading; Markov decision; deep reinforcement learning; federated learning; dueling network



Academic Editor: Mehdi Sookhak

Received: 12 July 2025

Revised: 24 August 2025

Accepted: 26 August 2025

Published: 1 September 2025

Citation: Zhao, H.; Li, Y.; Pang, Z.; Ma, Z. Federated Multi-Agent DRL for Task Offloading in Vehicular Edge Computing. *Electronics* **2025**, *14*, 3501. <https://doi.org/10.3390/electronics14173501>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the advancement of vehicle intelligence, high-real-time tasks such as autonomous driving and automatic navigation require rigorous service quality standards [1,2]. Conventional cloud computing services face challenges in achieving millisecond-level response times. Vehicle Edge Computing (VEC) mitigates this issue by allocating computing resources to Road Side Units (RSUs) [3,4], which theoretically facilitates substantial decreases in service response times. In resource-constrained VEC systems, edge nodes face challenges in evenly distributing high-concurrency, heterogeneous computational demands. The ambiguity of task requirements and the critical nature of latency characteristics present considerable challenges to the performance of task offloading [5,6]. Centralized optimization methods rely on the sharing of global states, which necessitates the transmission of sensitive information, including vehicle trajectories, thereby presenting a risk of privacy leakage. The achievement of efficient, secure, and reliable task offloading in dynamic environments presents a significant challenge that limits the effectiveness of vehicle-road collaboration.

Deep Reinforcement Learning (DRL) serves as an effective method for optimizing task offloading and resource allocation in dynamic vehicle networks [7–11], primarily due to its capacity for policy optimization independent of predefined models. Nonetheless,

DRL-based methods encounter challenges in managing task offloading within dynamic network environments that necessitate additional focus. Quality of Experience (QoE) is a dynamic performance metric that indicates user satisfaction, necessitating a balance between task processing latency and terminal energy consumption constraints. While some studies have investigated the trade-off between delay and energy consumption, they have insufficiently addressed user needs and have failed to meet QoE requirements [12–15]. Additional studies have employed a reinforcement learning framework utilizing deep Q-networks to concurrently optimize task offloading and resource allocation [16,17].

The combination of FL and DRL has gradually become an important research direction for task offloading and resource management in V2X networks. In [18], a task offloading scheme combining federated deep reinforcement learning with power control was proposed. This method achieves joint optimization of power control and task allocation in a vehicle edge computing environment, thereby improving system performance while reducing energy consumption. In [19], an online task offloading and resource allocation method based on federated deep reinforcement learning was proposed to address the task offloading problem in mobile edge computing under harsh environments. This method significantly enhances the robustness and adaptability of the system in complex dynamic network environments. In [20], focusing on the 6G-V2X network scenario, a task offloading and resource management framework combining federated learning is proposed. This framework addresses the challenges of computational resource scheduling and load balancing in large-scale V2X networks. Additionally, ref. [21] investigates V2X edge intelligence based on federated learning and proposes an adaptive parallel design to enhance training efficiency, thereby providing new insights for intelligent task offloading and resource optimization in V2X environments. In [22], a multi-objective evolutionary algorithm (SONG) is proposed for facility localization problems in vehicle-based fog computing networks, effectively reducing network energy consumption while maintaining task execution efficiency. In [23], a task-driven, priority-aware offloading scheme based on deep reinforcement learning technology was proposed. This method focuses on optimizing task offloading decisions based on task priority and deadline, thereby improving task completion efficiency while minimizing energy consumption. The joint optimization problem of task offloading and resource allocation is an NP-hard problem. Although existing research has made significant progress in this field, optimizing offloading decisions remains challenging, and conventional DRL algorithms often struggle to identify the global optimal solution for such problems. Especially in complex V2X environments, the strategies of various devices continue to evolve during training, which may lead to competition for resources such as bandwidth, computing power, and storage [24], thereby affecting the overall system performance.

A multi-agent DRL algorithm is proposed to optimize task offloading decisions in complex environments to address these issues. MAD3QN effectively mitigates the effects of non-stationarity and resource limitations in single-agent settings. The algorithm improves the stability of state value evaluation by decomposing Q-values into state values and action advantages [25]. Implementing FL helps address the challenges posed by non-independent and identically distributed (Non-IID) data during model training, while safeguarding privacy and improving overall model efficacy. Non-IID data is common in car edge computing, as many vehicles exhibit varied task kinds, computational capabilities, and data distributions, potentially resulting in inconsistent gradients and sluggish convergence during model training. This study presents the FL technique, which successfully alleviates the adverse impacts of non-IID data via distributed training and weighted aggregation. As a distributed machine learning approach, federated learning enables agents to share

solely the trained model parameters during the training process. This paper presents the following key contributions:

- For VEC situations, develop communication, computation, and system cost models predicated on task processing deadlines and energy limitations, and articulate the task offloading problem as a Markov Decision Process (MDP). This approach takes into account the dynamic attributes of RSU and MEC server workloads, formulates an effective reward mechanism, and emphasizes enhancing performance in MEC systems.
- To mitigate the expenses associated with prolonged task processing, we developed a task offloading approach utilizing multi-agent deep reinforcement learning. This approach employs a dual network design to partition the Q-function into a state value function and an action advantage function, hence augmenting the program's learning efficacy in intricate situations and facilitating the agent's ability to make optimal offloading decisions.
- The MAD3QN algorithm employs a distributed training approach that necessitates the handling of substantial data transport. Federated learning was implemented to enhance training efficiency. During this process, agents exchange training parameters following model aggregation and updates, which mitigates the effects of non-IID data while safeguarding data privacy and security, thereby enhancing the convergence rate and optimization efficacy of the model.

2. System Model

Figure 1 illustrates a vehicle traffic scenario on urban roads, featuring a VEC system that includes vehicles, RSUs, Mobile Edge Computing (MEC) servers, and base stations. Deploying MEC servers in proximity to RSUs is essential for enhancing the computational resource capacity and, consequently, the quality of service for vehicles. Each MEC server implements an agent utilizing deep reinforcement learning. Agents obtain revised global model parameters from base stations to enhance their local model training. Post-training, the models are disseminated to adjacent RSUs through wireless connections. RSUs utilize the trained models to determine optimal offloading decisions for each vehicle. It is assumed that all RSUs are administered by base stations and linked to RSU facilities through fiber optic cables. Let $\mathbb{I} = \{1, 2, \dots, I\}$ represent the set of cars, and $\mathbb{R} = \{1, 2, \dots, R\}$ designate the set of RSUs located along the roadside. Let $\mathcal{T} = \{1, 2, \dots, T\}$ denote the set of T time intervals, each enduring τ seconds. Each vehicle evaluates two distinct queues: the computational queue and the transmission queue. During time slot $t \in \mathcal{T}$, the RSU acquires task requests from vehicles within its coverage area and determines binary offloading based on the vehicles' status information. Designate the RSU coverage radius as r and the CPU clock frequency as f_{rsu} . Tasks are arranged according to a first-in, first-out methodology for local processing on the vehicle or designated for processing at the RSU. The RSU scheduler is responsible for allocating newly arrived tasks to each queue at the start of the time slot. Assume that each RSU comprises I queues, with each queue representing a vehicle $i \in \mathbb{I}$. Upon arrival at the RSU, a task is assigned to the appropriate queue. Let $z_i(t)$ denote the index of the computational work arriving at the RSU during time slot $t \in \mathcal{T}$. Let $\lambda_i(t)$ be the magnitude of the task, quantified in bits. The magnitude of task $z_i(t)$ corresponds to a discrete set $\eta = \{\lambda_1, \lambda_2, \dots, \lambda_i\}$, whereby $\lambda_i(t) \in \eta \cup \{0\}$ denotes the scenario in which no task is received. Let $\rho_i(t)$ denote the processing density of the task, indicating the number of CPU cycles required for a unit to perform (1000 cycles per bit). Let $\delta_i(t)$ represent the task's deadline, indicating that the task must be finalized before this time frame; otherwise, it will be forfeited. Let the binary variable $x_i(t)$ represent the offloading

choice, indicating whether the task is allocated to the computation queue ($x_i(t) = 0$) or the transmission queue.

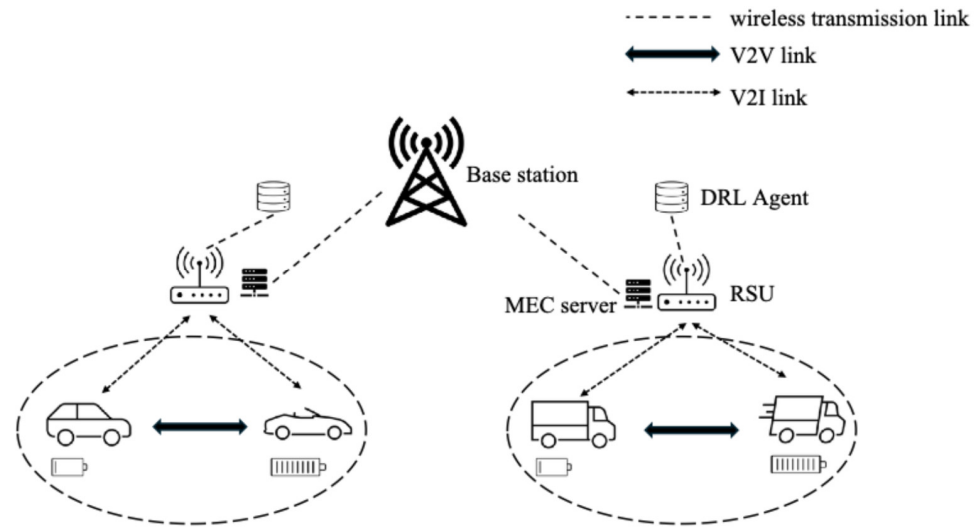


Figure 1. System model.

2.1. Communication Model

The RSU persistently gathers real-time vehicle status data, including task queue, vehicle speed, position, and available wireless resources. Every wireless communication link utilizes Orthogonal Frequency Division Multiple Access (OFDMA) [26] technology to provide communication among vehicles and between vehicles and infrastructure. Assuming the transmission channels possess perfect channel state information and that the vehicle's wireless channels stay stable during task transmission. In the vehicle-to-RSU channel, tasks in the transmission queue are allocated to the corresponding RSU using the wireless communication link. In time slot $t \in \mathcal{T}$, if task $z_i(t)$ is allocated to the transmission queue for offloading, let $t_i^{tran}(t) \in \mathcal{T}$ denote the time slot in which the task is assigned to the RSU or rejected. Let $T_i^t(t)$ represent the number of time slots the task $z_i(t)$ remains in the transmission queue; the vehicle computes the value of $T_i^t(t)$ before task offloading.

$$T_i^t(t) = \left\lceil \max_{t' \in \{0, 1, \dots, t-1\}} t_i^{tran}(t') - t + 1 \right\rceil \quad (1)$$

$t_i^{tran}(t')$ denotes the timeframe before which other tasks in the transmission queue were allocated or abandoned. If, during time slot $t \in \mathcal{T}$, the task $z_i(t)$ of vehicle $i \in \mathbb{I}$ is allocated to the transmission queue, then in the subsequent time slot $t_i^{tran}(t)$, the task is either scheduled or rejected.

$$t_i^{tran}(t) = \min \{t + T_i^t(t) + T_{i,r}^{tran} - 1, t + \delta_i(t) - 1\} \quad (2)$$

$T_{i,r}^{tran}$ denotes the communication latency necessary for transmitting task $z_i(t)$ from vehicle I_i to RSU R_r . Let $r_{i,r}$ denote the transmission rate from I_i to R_r . Shannon's formula articulates the communication rate of the uplink task from I_i to R_r as follows:

$$r_{i,r} = W \log_2 \left(1 + \frac{P_{i,r} H_{i,r}}{N_0 W + \sigma} \right) \quad (3)$$

$H_{i,r}$ represents the channel gain, while $P_{i,r}$ denotes the communication transmission power from the vehicle to the RSU. The channel capacity is W , and the Gaussian white noise power

is N_0W , where σ denotes co-channel interference. Consequently, the communication latency and transmission energy expenditure of task $z_i(t)$ from vehicle I_i to R_r are articulated as

$$T_{i,r}^{trans} = \frac{\lambda_{i,r}(t)}{r_{i,r}\tau} \quad (4)$$

$$E_{i,r}^{trans}(t) = T_{i,r}^{trans}(t) \cdot P_{i,r} \cdot \tau \quad (5)$$

2.2. Computation Model

Computational tasks may be performed locally on the vehicle or at the RSU. Define the binary variable $y_{i,r}(t)$ to denote the offloading target, showing if the work is offloaded to the RSU; assign $y_{i,r}(t) = 1$ accordingly, otherwise assign $y_{i,r}(t) = 0$. Given the brevity of the time slot t , it may be presumed that the quantity of vehicles $n_i(t)$ monitored by the RSU during this interval remains constant, represented as

$$n(t) = n_{max} \left(1 - \frac{s(t)}{s_{lim}} \right) \quad (6)$$

where n_{max} denotes the maximum capacity of vehicles that may be accommodated in the area, and $s(t)$ and s_{lim} signify the average speed and maximum speed limit of vehicles on the route, respectively.

2.2.1. Local Execution

Simulate a queuing system comprising a computational queue and a vehicle processor. At the commencement of time slot t , task $z_i(t)$ of vehicle I_i is allocated to the local computing queue. Let $t_i^c(t)$ be the time interval during which task $z_i(t)$ is executed or abandoned. If $t_i^c(t) = 0$, then the computational queue is empty. Let $T_i^c(t)$ be the number of time slots that the task must wait for processing in the computing queue.

$$T_i^c(t) = \left\lceil \max_{t' \in \{0,1,\dots,t-1\}} t_i^c(t') - t + 1 \right\rceil \quad (7)$$

$t_i^c(t')$ denotes the time slot t preceding which other tasks in the computation queue were executed or abandoned. If task $z_i(t)$ is allocated to the compute queue for processing during time slot t , then $t_i^c(t)$ is denoted as

$$t_i^c(t) = \min \left\{ t + T_i^c(t) + T_i^{local}(t) - 1, t + \delta_i(t) - 1 \right\} \quad (8)$$

If task $z_i(t)$ is not finalized before the conclusion of time slot $t + \delta_i(t) - 1$, the task is promptly abandoned. $T_i^{local}(t)$ denotes the computing latency necessary for task processing within the queue. Let f_i represent the processing capability of the vehicle I_i , and c denote the number of CPU cycles required to process a single bit of data; thus, the local computational delay of the vehicle is

$$T_i^{local}(t) = \frac{c \cdot \lambda_i(t) \cdot \rho_i(t)}{f_i \cdot n_i(t)} \cdot (1 - y_{i,r}(t)) \quad (9)$$

P_i^c denotes the CPU frequency used by vehicle I_i during task execution. The energy usage of vehicle I_i within the time interval t is articulated as

$$E_i^{local}(t) = T_i^{local}(t) \cdot P_i \cdot \tau \quad (10)$$

2.2.2. Edge Execution

Each RSU possesses several active queues and performs multiple tasks concurrently. The computational resources of the RSU fluctuate over time, and the computational capability of R_r during time slot t is $f_r(t)$. Assume that vehicle I_i allocates task $z_i(t')$ to RSU R_r during time slot $t' < t$. Let $z_{i,r}^R(t)$ denote the unique index of the task, $\lambda_{i,r}^R(t)$ represent the size of the task in active queue of R_r , $v_{i,r}^R(t)$ indicate the length of the active queue after the time slot, and $\varepsilon_{i,r}(t)$ signify the number of bits of discarded tasks in the active queue at the end of the time slot. Let $B_r(t)$ the collection of active queues of RSU R_r during time slot t .

$$B_r(t) = \left\{ i \mid i \in \mathbb{I}, r_{i,r}^R(t) > 0 \text{ or } v_{i,r}^R(t-1) > 0 \right\} \quad (11)$$

$$v_{i,r}^R(t) = \left[v_{i,r}^R(t-1) + \lambda_{i,r}^R(t) - \frac{f_r(t)\tau}{\rho_i(t)B_r(t)} - \varepsilon_{i,r}(t) \right] \quad (12)$$

Let $t_{i,r}^R(t)$ be the time interval during which task $z_{i,r}^R(t)$ is either executed or abandoned following its transfer to the active queue. If $t_{i,r}^R(t) = 0$, then the active queue is empty. Let $T_{i,r}^R(t)$ denote the number of time slots that task $z_{i,r}^R(t)$ must wait before being processed in the current queue.

$$T_{i,r}^R(t) = \left\lceil \max_{t' \in \{0,1,\dots,t-1\}} t_{i,r}^R(t') - t + 1 \right\rceil \quad (13)$$

$t_{i,r}^R(t')$ denotes the time slot during which tasks in the active queue are either processed or rejected before time slot t . During the time slot $t_{i,r}^R(t)$, the task is either executed or discarded.

$$t_{i,r}^R(t) = \min \left\{ t + T_{i,r}^R + T_r^{rsu}(t) - 1, t + \delta_i(t) - 1 \right\} \quad (14)$$

If the conclusion of slot $t + \delta_i(t) - 1$ does not finalize task $z_{i,r}^R(t)$, it is promptly discarded. Furthermore, each RSU performs tasks alone and does not distribute computational load with neighboring nodes. Upon execution of task $z_{i,r}^R(t)$ on its designated RSU, the requisite computational latency and energy consumption are as follows:

$$T_r^{rsu}(t) = \frac{c \cdot \lambda_{i,r}^R(t) \cdot \rho_i(t) \cdot B_r(t)}{f_r(t) \cdot n_i(t)} \quad (15)$$

$$E_r^{rsu}(t) = T_r^{rsu}(t) \cdot P_r \cdot \tau \quad (16)$$

P_r represents the CPU frequency utilization when RSU R_r performs the task. Furthermore, take into account the energy expenditure while the vehicle I_i awaits mission completion and return, articulated as

$$E_r^{wait}(t) = T_r^{rsu}(t) \cdot p_r^{wait} \cdot \tau \quad (17)$$

The whole energy consumption of task $z_i(t')$ performed at the edge is articulated as

$$E_i^{total}(t) = E_{i,r}^{trans}(t) + \sum_r E_{i,r}^{rsu}(t) + E_i^{wait}(t) \quad (18)$$

3. Problem Formulation and Algorithm Design

3.1. Problem Formulation

The system model aims to enhance task offloading decisions, thereby reducing processing costs associated with latency, energy consumption, and task loss rates, and ultimately elevating vehicle service quality. The vehicle model takes into account the battery power, which has become a crucial issue in the era of new energy cars. Before executing the ideal offloading option, the agent assesses the vehicle's status and battery power, and then

determines the optimal selection upon arrival of a new job. The expense associated with task processing can be articulated as

$$w_i(t) = \phi_i(t)T_i(t) + (1 - \phi_i(t))E_i(t) \quad (19)$$

$\phi_i(t)$ denotes the battery power of the vehicle. $T_i(t)$ denotes the delay of the task following the agent's decision—the cumulative duration of time slots from time slot t until the task is executed. Upon task abandonment, $T_i(t) = 0$. $E_i(t)$ represents the energy consumption of the task after the agent has made its decision. Mathematically expressed as

$$T_i(t) = (1 - x_i(t))(t_i^c(t) - t + 1) + x_i(t) \left(\sum_r \sum_{t'=t}^T (z_{i,r}^R(t') = z_i(t)) t_{i,r}^R(t') - t + 1 \right) \quad (20)$$

$$E_i(t) = (1 - x_i(t))E_i^{local}(t) + x_i(t) \left(\sum_r \sum_{t'=t}^T (z_{i,r}^R(t') = z_i(t)) E_i^{total}(t) \right) \quad (21)$$

3.2. Markov Decision Process

To reduce the long-term task processing costs of the system, it is essential to transform this optimization issue into a Markov decision process, thereby elucidating the decision-making process of task offloading. Subsequently, we shall elucidate the state space, action space, and reward function within the Markov decision process.

State space: At the commencement of time slot t , the agent gathers vehicle status data within the vicinity, denoted as vector $s_i(t)$, including the magnitude of freshly arriving tasks, battery power, queue information, and RSU workload history.

$$s_i(t) = (\lambda_i(t), T_i^t(t), T_i^c(t), v_i^R(t-1), \phi_i(t), H(t)) \quad (22)$$

where $v_i^R(t-1)$ is the queue duration in the preceding time slot of R_r , and $\phi_i(t)$ represents the battery power of vehicle I_i during time slot t . To forecast the future workload of RSU, $H(t)$ is depicted as historical load data. This data is documented in the $T^s \times J$ matrix for the time interval T^s , spanning from $t - T^s$ to $t - 1$. For R_r in the i^{th} time slot of $T^s - t$, $h_{i,t}(t)$ is delineated as

$$h_{i,t}(t) = B_r(t - T^s + i - 1) \quad (23)$$

Action space: Exemplifies the conduct and choices of agents. Let $a_i(t)$ denote the action executed by vehicle I_i during time slot t . This step encompasses the choice to unload the task and the identification of the unloading node. Among these, $y_i(t) = y_{i,r}(t)$ is the designated RSU for the unloading task.

$$a_i(t) = (x_i(t), y_i(t)) \quad (24)$$

Reward function: Characterize the task-unloading choice of vehicle I_i as a function that maps its state to the associated action, denoted as $\Pi_i : S \rightarrow A$. Let $q_i(s_i(t), a_i(t))$ denote the reward function corresponding to task $z_i(t)$. Considering the observed state $s_i(t)$ and the chosen action $a_i(t)$, the reward function is denoted as

$$q_i(s_i(t), a_i(t)) = \begin{cases} \mathbb{R} - w_i(s_i(t), a_i(t)) & \text{C1} \\ -E_i(s_i(t), a_i(t)) & \text{C2} \end{cases} \quad (25)$$

where C1 signifies the execution of task $z_i(t)$, C2 indicates the rejection of task $z_i(t)$; $w_i(s_i(t), a_i(t))$ represents the cost incurred when opting for action $a_i(t)$ in state $s_i(t)$;

$E_i(s_i(t), a_i(t))$ depicts the energy expenditure associated with selecting action $a_i(t)$ in state $s_i(t)$; $\mathbb{R} > 0$ indicates the fixed reward for task completion, if $z_i(t) = 0$, then $q_i(s_i(t), a_i(t)) = 0$. In the following sections, the reward function $q_i(s_i(t), a_i(t))$ is referred to as $q_i(t)$. In the observed environmental state $s_i(t) \in S$, the vehicle executes action $a_i(t) \in A$ by policy Π_i . The agent seeks to identify the optimal policy Π_i^* , to minimize long-term task processing costs.

$$\mathbb{E}_i^* = \operatorname{argmax}_{\Pi_i} \mathbb{E} \left[\sum_{t \in \mathcal{T}} \gamma^{t-1} q_i(t) | \Pi_i \right] \quad (26)$$

$\gamma \in (0, 1)$ represents the discount factor, and the predicted $\mathbb{E}$ considers the time-dependent system environment. To address the optimization challenge presented in Formula (26) inside a dynamic environment, we propose a Deep Reinforcement Learning (DRL) optimization approach to ascertain the relationship between each state-action combination and the long-term task processing cost.

Figure 2 illustrates that each agent is outfitted with a neural network including five layers: an input layer, a dense layer, an advantage value layer, and an output layer. Let θ_i represent the parameter vector of the neural network, which is utilized to preserve the connection weights and neuron biases across all layers. The input layer consists of six neurons that receive state information, with each neuron representing a specific parameter, which is subsequently transmitted to the hidden layer. The dense layer comprises a collection of neurons employing the rectified linear unit (ReLU) as the activation function. The initial dense layer establishes connections from the input layer's neurons to each neuron in the dense layer, which subsequently connects to each neuron in the following dense layer. In the second dense layer, the output of each neuron is interconnected with all neurons in the A&V layer. The A&V layer is a novel element introduced by Dueling DDQN into the neural network, comprising two separate dense networks: the action advantage vector (A) and the state value (V). The neural network independently learns these outputs, which are then aggregated through summation to obtain the Q-value. For each agent, let $V_i(s_i(t); \theta_i)$ represent the state value and $A_i(s_i(t), a'; \theta_i)$ designate the action advantage value of action $a_i(t) \in A$ in the context of state $s_i(t) \in S$. The parameter vector θ_i is responsible for ascertaining these values and is modified during the training of the MAD3QN algorithm. The A&V layer and the output layer together establish $Q_i(s_i(t), a; \theta_i)$, namely

$$Q_i(s_i(t), a; \theta_i) = V_i(s_i(t); \theta_i) + \left(A_i(s_i(t), a; \theta_i) - \frac{1}{|A|} \sum_{a' \in A} (A_i(s_i(t), a'; \theta_i)) \right) \quad (27)$$

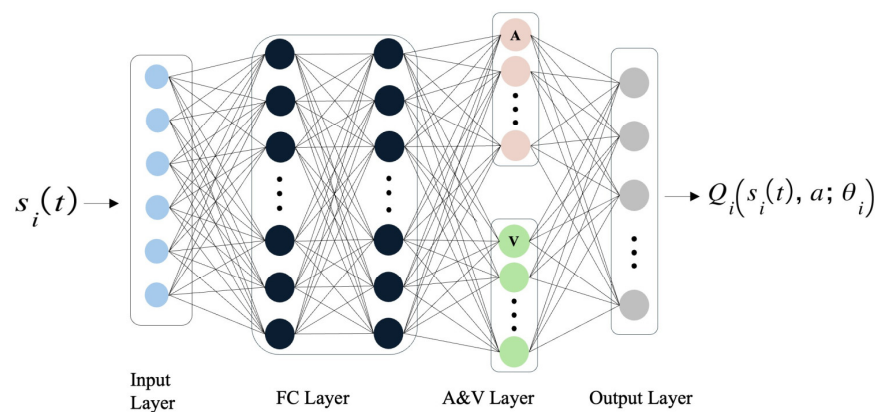


Figure 2. Neural network structure of Dueling DDQN.

3.3. MAD3QN Optimization with Enhanced Q-Values

In this section, the proposed MAD3QN algorithm effectively minimizes task processing costs. Based on the dueling network architecture, MAD3QN consists of two key components: the evaluation network (Net_i^E) and the target network (Net_i^T). These components share the same neural network structure but differ in their parameter vectors, θ_i^E and θ_i^T . Net_i^E is responsible for real-time learning and decision offloading, taking the current state, $s_i(t) \in S$, as input and outputting the Q-value, $Q_i^E(s_i(t), a; \theta_i^E)$. Net_i^E selects the optimal action $a_i(t)$, based on the maximum Q-value. The target Q-value of Net_i^T is $Q_i^T(s_i(t), a; \theta_i^T)$, representing the long-term reward caused by a specific action under the observed state. The target Q-value serves as a reference for updating the network parameter vector θ_i^E , which is achieved by minimizing the difference between the Q-values in Net_i^E and Net_i^T . The replay experience buffer is responsible for recording observed experiences, $(s_i(t), a_i(t), q_i(t), s_i(t+1))$. In Algorithm 1, each agent initializes the experience replay buffer D using neural networks Net_i^E and Net_i^T . At the start of each iteration, the initial state $s_i(1)$ is set, and an update parameter request is generated. The agent selects the following action using an ε -greedy strategy:

$$a_i(t) = \begin{cases} \arg \max_{a \in \mathcal{A}} Q_i^E(s_i(t), a; \theta_i^E), & w.p. 1 - e \\ a \text{ random action from } \mathcal{A}, & w.p. e \end{cases} \quad (28)$$

In the ε -greedy strategy, a random action is selected with probability $(w.p.)e$; otherwise, the agent chooses the action with the highest Q-value based on the assessment network settings. Following the evaluation network's update of the parameter vector, this experience is recorded in D. For each experience $d_i \in d$, it computes Q'_{i,d_i} , denoting the reward value of experience d_i and the discounted Q-value of the action executed by the neural network Net_i^E in the subsequent state of experience d_i , i.e.,:

$$Q'_{i,d_i} = q_i(d_i) + \gamma Q_i^T(s_i(d_i + 1), a_{d_i}; \theta_i^T) \quad (29)$$

where a_{d_i} denotes the ideal action selection determined by the maximum Q-value in Net_i^E under state $s_i(d_i + 1)$, i.e.,:

$$a_{d_i} = \arg \max_{a \in \mathcal{A}} Q_i^E(s_i(d_i + 1), a; \theta_i^E) \quad (30)$$

This value reflects the reward noted in d_i and the estimated projected reward value. The agent trains its neural network using previous experience samples, concurrently updating the parameters θ_i^E of the evaluation network and computing the vector $Q'_i = (Q'_{i,d_i}), d_i \in d$. The update is performed by calculating the loss function through evaluating the Q-values of both the network and the target network. The loss function is as follows:

$$L(\theta_i^E, Q'_i) = \frac{1}{|d|} \sum_{d_i \in d} (Q_i^E(s_i(d_i), a_i(d_i); \theta_i^E) - Q'_{i,d_i}) \quad (31)$$

Algorithm 1: MAD3QN training process

```

1: Input: Observed environmental state  $s_i(t)$ ; Output: parameters  $\theta_i^E$ 
2: Initialization: empirical replay buffer D, evaluation network  $Net_i^E$  and target  $Net_i^T$ ,
   parameters  $\theta_i^E$  and  $\theta_i^T$ 
3: for episode  $i = 1, 2, \dots, M$  do
4:   Initialize the state  $s_i(t)$ 
5:   for each agent  $i$  in a time slot  $t \in \mathcal{T}$  do
6:     According to the current state  $s_i(1)$ , randomly select an action or choose to
        $a_i(t) = \arg \max_{a \in \mathcal{A}} Q_i^E(s_i(t), a; \theta_i^E)$ 
7:     Perform the action  $a_i(t)$  to obtain  $s_i(t+1)$  and the reward  $q_i(t)$ 
8:     Store the experience  $(s_i(t), a_i(t), q_i(t), s_i(t+1))$  in D.
9:     for agent  $i = 1$  do
10:      Obtain a set of experience samples  $d$  from D.
11:      for each experience  $d_i \in d$  do
12:        Get the experience  $(s_i(d_i), a_i(d_i), q_i(d_i), s_i(d_i+1))$ 
13:        Calculate according to (29)  $Q'_{i,d_i}$ 
14:      end for
15:      Setting  $Q'_i = (Q'_{i,d_i}), d_i \in d$ 
16:      Update parameters  $\theta_i^E$  by minimizing the loss function  $L(\theta_i^E, Q'_i)$  (Equation (31))
17:      Update parameters for each agent  $Q_i^T = Q_i^E$ 
18:    end for
19:  if end of episode  $i$  then
20:    Update target network parameters:
21:    Perform model aggregation
22:  end if
23: end for
24: end for

```

3.4. Complexity Analysis

The amount of experience needed to find the best offloading approach determines MAD3QN's computational complexity. Each training event requires backpropagation computations with a computational complexity of $O(C)$, where C is the number of neural network multiplication operations. In each training iteration initiated by a new task, a sample set of $|d|$ experiences is extracted from D. The training method incorporates d^{ep} training sets, each including K predicted tasks, resulting in a total computational complexity of $O(d^{ep}K|d|C)$, characterized by polynomial complexity.

3.5. FL-Based Optimization Method for MAD3QN Models

During model training, frequent data transfers significantly consume network bandwidth, complicating resource allocation and increasing the risk of data leakage during transmission. To resolve these challenges, FL technology is utilized during model training. Unlike traditional dispersed training, federated learning facilitates collaborative model training while safeguarding data privacy and security, improving model performance. In federated learning, the heterogeneity of data distribution among various devices, such as automobiles, roadside units, or mobile edge computing servers, frequently renders gradient consistency a critical aspect influencing model stability and convergence. Each agent trains the model with local input, and the computed gradients may disagree, resulting in instability and protracted convergence during the global model update process. This study utilizes a weighted average method for model parameter aggregation to alleviate the effects

of non-IID data on gradient consistency, and implements a soft update strategy to facilitate gradual and smooth convergence of the global model.

As illustrated in Figure 3, during the model initialization phase, the MEC server acquires the global MAD3QN model $M(t)$ from the base station and sets the model parameters. Each MEC server utilizes local data to conduct preliminary training on model $M_1(t), M_2(t), \dots, M_i(t)$ during the training process. Upon the completion of local training, the revised model parameters, instead of the original data, are transmitted to the base station. The base station conducts weighted computations on the model according to the ratio of each MEC server's local data utilization to the total data to achieve the updated global model $M(t+1)$. The revised model parameters are subsequently communicated to each MEC server for the forthcoming training cycle. Upon acquisition of the new model, the MEC servers execute a soft update of the model parameters to mitigate the influence of extraneous parameters on the overall model. This federated learning approach guarantees data privacy while enhancing model stability and convergence velocity by tackling challenges like data distribution heterogeneity and gradient inconsistency among agents. The equations for local updates and global aggregation are as follows:

$$M_i(t) = \alpha M_i(t) + (1 - \alpha)M(t) \quad (32)$$

$$M(t+1) = \sum_{i=1}^N \frac{N_i}{N} M_i(t) \quad (33)$$

where α denotes the soft update weight, N represents the overall number of training samples, and N_i represents the quantity of locally trained samples associated with the MEC server. **Algorithm 2** presents the pseudocode for the MAD3QN model aggregation approach grounded in FL.

Algorithm 2: FL-based MAD3QN model training

- 1: Input: Global model parameters $M(t)$; Local model parameters $M_i(t)$
 - 2: Output: updated global model parameters $M(t+1)$
 - 3: Initialization: In time slot $t \in \mathcal{T}$, the base station initializes the MAD3QN model $M(t)$; each agent initializes the local
 - 4: MAD3QN model $M_i(0)$.
 - 5: for round $i = 1, 2, \dots, M$ do
 - 6: for agent $i = 1$ do
 - 7: Download global parameters from base station $M(t)$
 - 8: Update local parameters according to (32) $M_i(t)$
 - 9: Perform local training based on local data
 - 10: Update the trained parameters $M_i(t+1)$ and upload to the base station
 - 11: end for
 - 12: Base station aggregates the uploaded by the agents according to Equation (33) $M_i(t)$
 - 13: Each smart body receives the updated weighted average parameters $M_i(t+1)$
 - 14: end for
-

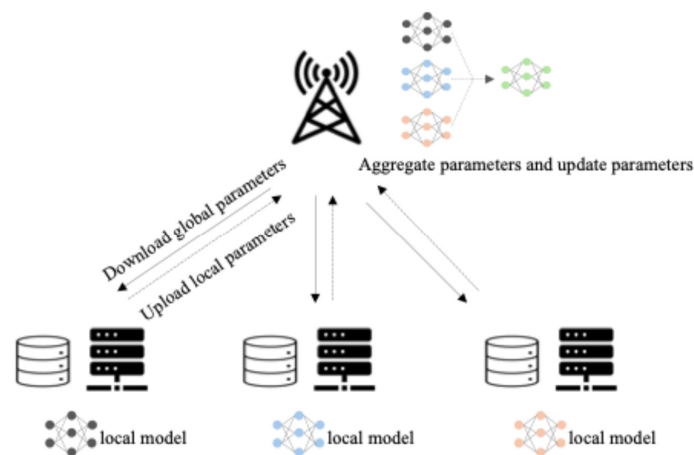


Figure 3. Federated learning training process.

4. Experimental Results

4.1. Parameter Settings

Consider an urban two-way road vehicle traffic scenario with 1 base station, 6–10 RSU devices, and 10–50 vehicles in the RSU coverage region. As per 3GPP R15 [27] and C-V2X vehicle-to-everything and vehicle-to-infrastructure (V2I) essential technologies [28], the primary simulation parameters [29] and hyperparameters are shown in Tables 1 and 2. The Net_i^E and Net_i^T components of each intelligent agent consist of two layers of fully connected neural networks, featuring 128 neurons in the first layer and 64 neurons in the second layer, employing ReLU as the activation function. The RMSProp optimizer is employed to train and adjust the weights of the neural network iteratively. The algorithm undergoes training throughout 10,000 episodes. Each episode comprises 100 time slots, each lasting 0.1 s. The learning rate is established at 0.001, and the discount factor is determined to be 0.9. The likelihood of random exploration commences at an initial value and progressively diminishes by 1, converging around 0.01. Simulation tests were conducted using Python 3.8.19 and TensorFlow 2.0.0 to assess the efficacy of the FL-MAD3QN algorithm in a VEC context.

Table 1. Hyperparameters of the FL-MAD3QN algorithm.

Parameter	Values
Learning rate α	0.001
Discount factor γ	0.9
Initial epsilon ϵ_0	1.0
Epsilon decay rate ϵ_{decay}	0.99
Batch size	32
Buffer size D	2000
Soft update weight	0.995

To assess the efficacy of the suggested algorithm, the subsequent algorithms were compared:

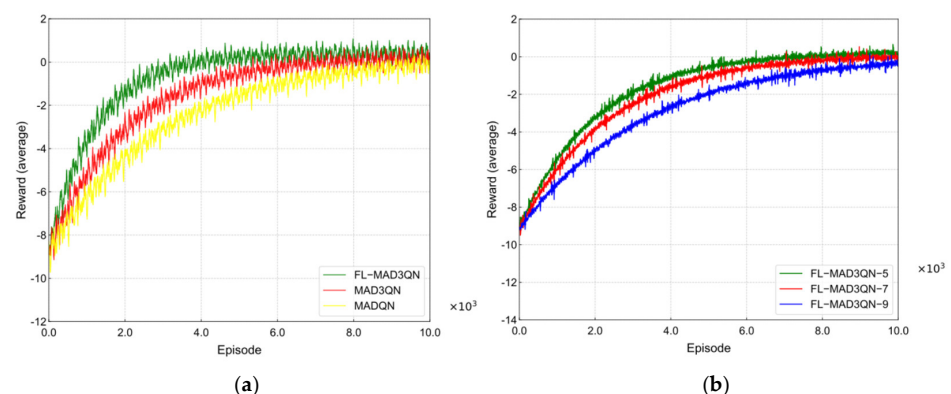
- Local computation (LC): Vehicles utilize their inherent computing capabilities to execute all computational operations.
- Full offloading (FO): All vehicle tasks are transferred to the RSU for remote execution.
- FL-DQN: Achieves the ideal offloading technique through integration with federated learning.
- DDPG: A task offloading algorithm utilizing an evaluation network and a target network architecture.

Table 2. Simulation parameters.

Parameter	Values
Radio Frequency	2 GHz
Vehicle Computing Resources	[0.1–1] GHz
Number of vehicles; Vehicle speed	[10, 50]; 10 m/s
Noise power	−120 dBm
MEC Server Computing Resources	40 GHZ
RSU Coverage	[100–300] m ²
Number of RSUs	6, 8, 10
Task Size	[1.0, 1.1, ..., 10.0] Mbits
Task deadline	10 time slots (1 s)
Vehicle power status	[20, 50, 80]
RSU calculation power and Bandwidth	10 W; 100 MHz
Vehicle Transmission Power and Standby Power	0.5 W; 0.1 W
Federal Learning Rate	0.4
V2I/V2V Transmit Power and Communication Bandwidth	0.2 W; 10 MHz

4.2. Convergence Analysis and Performance Comparison

Convergence Analysis: Figure 4 depicts the rewards of several algorithms and the corresponding benefits based on differing agent quantities within each algorithm. The convergence efficacy of the FL-MAD3QN method is compared with that of MAD3QN and MADQN Figure 4a. Initially, each agent trains its model, leading to minimal variations in average payouts. With the escalation of training cycles, the experience replay buffer data expands steadily, gradually enhancing and stabilizing the agents' average rewards. As illustrated Figure 4a, MAD3QN encounters substantial reward variability during the initial training phases, resulting in a reduced convergence rate in the absence of federated learning. This signifies that gradient inconsistency, resulting from non-IID input, impacts the model's training process. Conversely, following the implementation of federated learning, FL-MAD3QN significantly alleviates the effects of non-IID data during global model aggregation, resulting in more consistent updates among various agents. As a result, the global model exhibits accelerated convergence and diminished reward fluctuations. Following a specified number of training iterations, FL-MAD3QN initiates model aggregation, causing agents to diverge from local optima progressively. Consequently, the average reward escalates more consistently, and the convergence performance surpasses that of MAD3QN and MADQN without FL. Figure 4b, the convergence performance of FL-MAD3QN with varying agent quantities is illustrated. As the quantity of agents escalates, the dimensions of both the state space and action space expand, resulting in an initial decline in the average reward of FL-MAD3QN. Nevertheless, once the model converges and stabilizes, it continues to yield a satisfactory average reward.

**Figure 4.** (a) Rewards for different algorithms; (b) rewards for different numbers of agents.

Performance Comparison: Figure 5 illustrates a performance comparison across different task arrival rates. Figure 5a, the proposed methodology consistently outperforms current techniques as the task arrival rate fluctuates. As the task arrival rate increases, the performance advantage of the proposed approach becomes more apparent. At a task arrival rate of 300, the proposed technique achieves 18.06% and 58.29% higher task completions compared to FL-DQN and DDPG, respectively. Figure 5b, for a task arrival rate of 300, the proposed solution reduces the average delay by at least 10.12% and 18.20% relative to FL-DQN and DDPG, respectively. When the task arrival rate surpasses 500, the proposed algorithm may demonstrate elevated average latency, which is ascribed to its superior task completion rate relative to other methods. Figure 5c, At diminished task arrival rates, the total energy consumption across all techniques is similar. At a task arrival rate of 500, the proposed solution reduces total energy consumption by 9.08% compared to FL-DQN and by 13.72% relative to DDPG. The FL-MAD3QN approach utilizes extensive computational resources, while the LC and FO methods depend exclusively on local and edge node resources, respectively. Thus, their task completion rates are far lower than the FL-MAD3QN algorithm, resulting in lower energy consumption. Figure 5d, depicts the average reward at different task arrival rates. At a task arrival rate of 300, the proposed method achieves an average reward improvement of 18.25% and 38.78% compared to FL-DQN and DDPG, respectively.

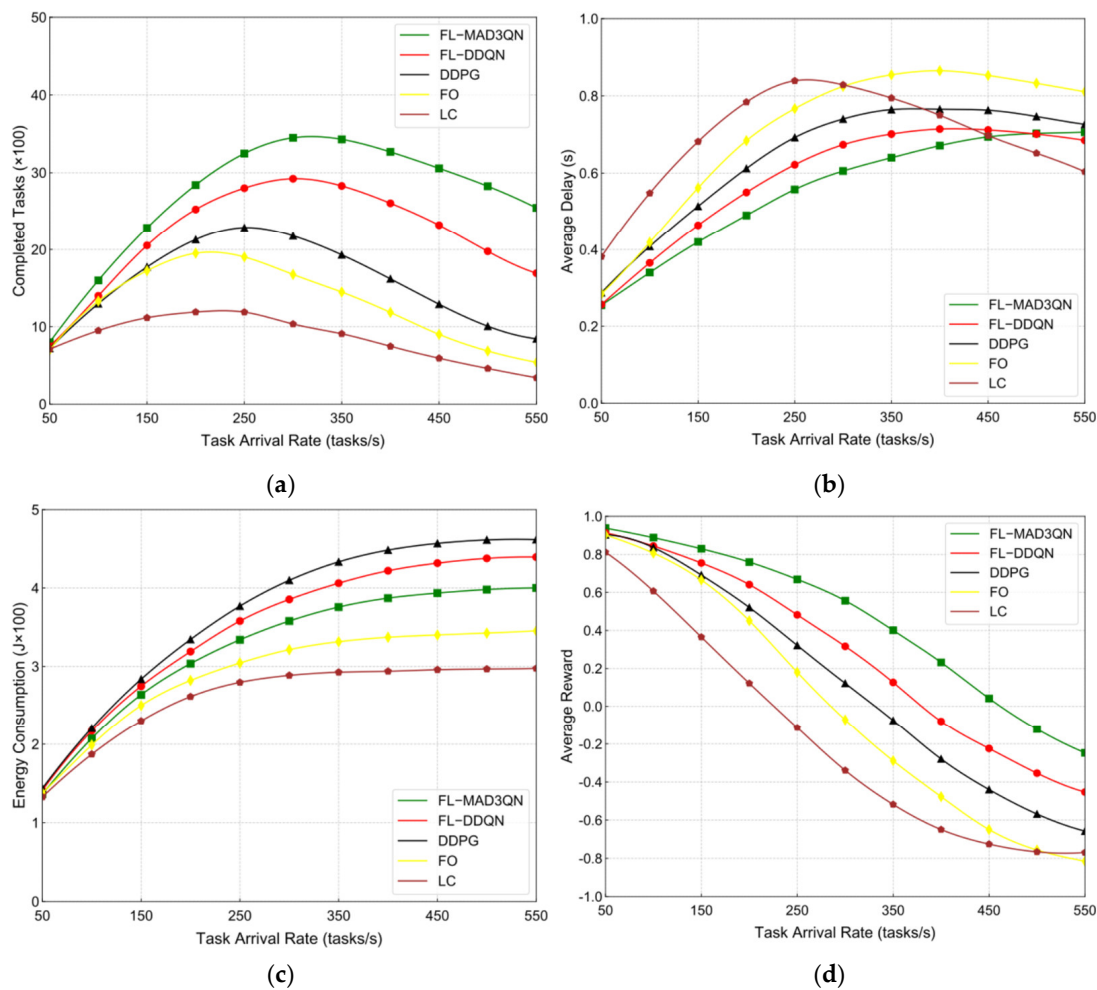


Figure 5. Performance comparison at different task arrival rates: (a) completed tasks; (b) average delay; (c) energy consumption; (d) average reward.

Figure 6a illustrates that with an increase in the number of cars, the suggested algorithm consistently outperforms alternative techniques in terms of task completion rate. Once the vehicle count reaches 50, the work requires considerable processing resources, enabling the agent to arrange tasks efficiently. The FL-MAD3QN algorithm attains a minimum of a 12.91% enhancement in task completion rate. Figure 6b indicates that the overall average delay has a tendency to increase. Upon reaching a vehicle count of 100, the FL-MAD3QN algorithm decreases the average delay by a minimum of 11%. LC exclusively executes tasks locally, resulting in a consistent average delay. Figure 6c, when the quantity of cars rises, the system's processing resources expand, resulting in an upward trajectory in total energy usage. The suggested technique decreases total energy usage by a minimum of 15.78% and 24.03% when compared to FL-DQN and DDPG, respectively. The FL-MAD3QN algorithm employs comprehensive computational resources, whereas LC and FO rely solely on local and edge nodes for task processing, resulting in reduced energy consumption. Figure 6d illustrates the average payoff as the quantity of vehicles escalates. Upon reaching a count of 100, the strain on edge nodes escalates, and the suggested algorithm achieves an average reward enhancement of at least 21% compared to FL-DQN and DDPG. As LC continually executes tasks locally, the average reward remains constant.

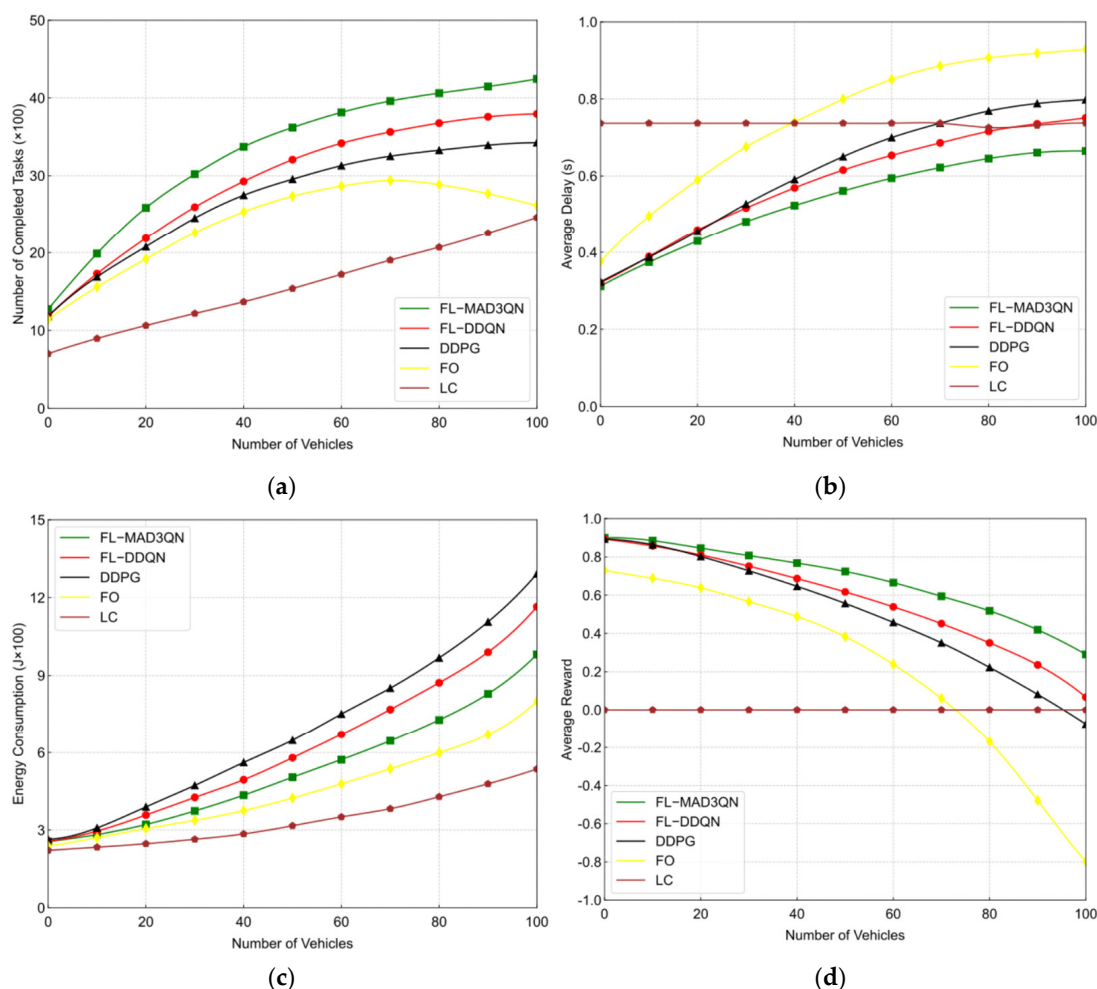


Figure 6. Performance comparison with different numbers of vehicles: (a) number of completed tasks; (b) average delay; (c) energy consumption; (d) average reward.

5. Conclusions

This study presents an enhanced deep reinforcement learning task offloading strategy within the context of vehicular edge computing to minimize latency and energy expenditure

while adhering to task processing deadlines and energy limitations. Initially, in light of the dynamic and unpredictable characteristics of mobile settings, communication and computational models are established, and the costs associated with task processing are evaluated. Secondly, utilizing the previously mentioned models, an optimization target is formulated, and the task offloading issue is represented as a Markov decision process. A method based on the MAD3QN framework is developed to reduce the system's task processing costs, allowing the agent to make optimal offloading decisions independently of other task models or agent offloading choices. Ultimately, FL is integrated with the MAD3QN framework to improve model training efficiency while safeguarding user data privacy. Experiments indicate that the proposed algorithm exhibits favorable performance in terms of model convergence, latency, energy consumption, and task completion rate.

Subsequent efforts will enhance the algorithm's efficacy by utilizing prioritized experience replay and implementing weight allocation techniques to improve the model's learning efficiency. Furthermore, the interrelations among tasks may be taken into account to enhance the task model.

Author Contributions: Conceptualization, H.Z.; methodology, H.Z.; software, Y.L.; validation, Y.L.; formal analysis, Y.L.; investigation, Z.P.; resources, Z.P.; data curation, Z.M.; writing—original draft preparation, Y.L.; writing—review and editing, Y.L.; visualization, Z.M.; supervision, Z.M.; project administration, H.Z.; funding acquisition, H.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China (42471213) and Northeast Geoscience and Science and Technology Innovation Center Regional Innovation Fund Project (QCJJ2023-49).

Data Availability Statement: Derived data supporting the findings of this study are available from the corresponding author on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Panigrahy, S.K.; Emany, H. A survey and tutorial on network optimization for intelligent transport system using the internet of vehicles. *Sensors* **2023**, *23*, 555. [\[CrossRef\]](#)
2. Gong, T.; Zhu, L.; Yu, F.R.; Tang, T. Edge intelligence in intelligent transportation systems: A survey. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 8919–8944. [\[CrossRef\]](#)
3. Luo, G.; Shao, C.; Cheng, N.; Zhou, H.; Zhang, H.; Yuan, Q.; Li, J. Edge Cooper: Network-aware cooperative LiDAR perception for enhanced vehicular awareness. *IEEE J. Sel. Areas Commun.* **2023**, *42*, 207–222. [\[CrossRef\]](#)
4. Yin, L.; Luo, J.; Qiu, C.; Wang, C.; Qiao, Y. Joint task offloading and resources allocation for hybrid vehicle edge computing systems. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 10355–10368. [\[CrossRef\]](#)
5. Dong, S.; Tang, J.; Abbas, K.; Hou, R.; Kamruzzaman, J.; Rutkowski, L.; Buyya, R. Task offloading strategies for mobile edge computing: A survey. *Comput. Netw.* **2024**, *254*, 110791. [\[CrossRef\]](#)
6. Ren, J.; Hou, T.; Wang, H.; Tian, H.; Wei, H.; Zheng, H.; Zhang, X. Collaborative task offloading and resource scheduling framework for heterogeneous edge computing. *Wirel. Netw.* **2024**, *30*, 3897–3909. [\[CrossRef\]](#)
7. Zabihi, Z.; Moghadam, A.M.E.; Rezvani, M.H. Reinforcement learning methods for computation offloading: A systematic review. *ACM Comput. Surv.* **2023**, *56*, 17. [\[CrossRef\]](#)
8. Luo, Z.; Dai, X. Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges. *Alex. Eng. J.* **2024**, *108*, 89–107. [\[CrossRef\]](#)
9. Xue, J.; Wang, L.; Yu, Q.; Mao, P. Multi-Agent Deep Reinforcement Learning-based Partial Offloading and Resource Allocation in Vehicular Edge Computing Networks. *Comput. Commun.* **2025**, *234*, 108081. [\[CrossRef\]](#)
10. He, H.; Yang, X.; Mi, X.; Shen, H.; Liao, X. Multi-Agent Deep Reinforcement Learning Based Dynamic Task Offloading in a Device-to-Device Mobile-Edge Computing Network to Minimize Average Task Delay with Deadline Constraints. *Sensors* **2024**, *24*, 5141. [\[CrossRef\]](#)
11. Zhao, P.; Kuang, Z.; Guo, Y.; Hou, F. Task offloading and resource allocation in UAV-assisted vehicle platoon system. *IEEE Trans. Veh. Technol.* **2024**, *74*, 1584–1596. [\[CrossRef\]](#)

12. Sun, G.; He, L.; Sun, Z.; Wu, Q.; Liang, S.; Li, J.; Niyato, D.; Leung, V.C.M. Joint task offloading and resource allocation in aerial-terrestrial UAV networks with edge and fog computing for post-disaster rescue. *IEEE Trans. Mob. Comput.* **2024**, *23*, 8582–8600. [\[CrossRef\]](#)
13. Farimani, M.K.; Karimian-Aliabadi, S.; Entezari-Maleki, R.; Egger, B.; Sousa, L. Deadline-aware task offloading in vehicular networks using deep reinforcement learning. *Expert Syst. Appl.* **2024**, *249*, 123622. [\[CrossRef\]](#)
14. He, H.; Yang, X.; Huang, F.; Shen, H.; Tian, H. Enhancing QoE in Large-Scale U-MEC Networks via Joint Optimization of Task Offloading and UAV Trajectories. *IEEE Internet Things J.* **2024**, *11*, 35710–35723. [\[CrossRef\]](#)
15. Zhu, K.; Li, S.; Zhang, X.; Wang, J.; Xie, C.; Wu, F.; Xie, R. An Energy-Efficient Dynamic Offloading Algorithm for Edge Computing Based on Deep Reinforcement Learning. *IEEE Access* **2024**, *12*, 127489–127506. [\[CrossRef\]](#)
16. Zhai, L.; Lu, Z.; Sun, J.; Li, X. Joint task offloading and computing resource allocation with DQN for task-dependency in multi-access edge computing. *Comput. Netw.* **2025**, *263*, 111222. [\[CrossRef\]](#)
17. Wu, J.; Du, R.; Wang, Z. Deep reinforcement learning with dual-Q and Kolmogorov–Arnold Networks for computation offloading in Industrial IoT. *Comput. Netw.* **2025**, *257*, 110987. [\[CrossRef\]](#)
18. Moon, S.; Lim, Y. Federated deep reinforcement learning based task offloading with power control in vehicular edge computing. *Sensors* **2022**, *22*, 9595. [\[CrossRef\]](#)
19. Xiang, H.; Zhang, M.; Jian, C. Federated deep reinforcement learning-based online task offloading and resource allocation in harsh mobile edge computing environment. *Clust. Comput.* **2024**, *27*, 3323–3339. [\[CrossRef\]](#)
20. Hasan, M.K.; Jahan, N.; Nazri, M.Z.; Islam, S.; Khan, M.A.; Alzahrani, A.I.; Alalwan, N.; Nam, Y. Federated learning for computational offloading and resource management of vehicular edge computing in 6G-V2X network. *IEEE Trans. Consum. Electron.* **2024**, *70*, 3827–3847. [\[CrossRef\]](#)
21. Qiang, X.; Chang, Z.; Ye, C.; Hämäläinen, T.; Min, G. Split federated learning empowered vehicular edge intelligence: Concept, adaptive design, and future directions. *IEEE Wirel. Commun.* **2025**, *32*, 90–97. [\[CrossRef\]](#)
22. Hussain, M.; Azar, A.T.; Ahmed, R.; Amin, S.U.; Qureshi, B.; Reddy, V.D.; Alam, I.; Khan, Z.I. SONG: A multi-objective evolutionary algorithm for delay and energy aware facility location in vehicular fog networks. *Sensors* **2023**, *23*, 667. [\[CrossRef\]](#)
23. Hao, H.; Xu, C.; Zhang, W.; Yang, S.; Muntean, G.-M. Task-Driven Priority-Aware Computation Offloading Using Deep Reinforcement Learning. *IEEE Trans. Wirel. Commun.* **2025**. [\[CrossRef\]](#)
24. Pang, S.; Wang, T.; Gui, H.; He, X.; Hou, L. An intelligent task offloading method based on multi-agent deep reinforcement learning in ultra-dense heterogeneous network with mobile edge computing. *Comput. Netw.* **2024**, *250*, 110555. [\[CrossRef\]](#)
25. Du, R.; Wu, J.; Gao, Y. Dual-Q network deep reinforcement learning-based computation offloading method for industrial internet of things. *J. Supercomput.* **2024**, *80*, 25590–25615. [\[CrossRef\]](#)
26. Tan, L.; Kuang, Z.; Zhao, L.; Liu, A. Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing. *IEEE Trans. Wirel. Commun.* **2021**, *21*, 1960–1972. [\[CrossRef\]](#)
27. Shen, J.; Du, Z.; Zhang, Z.; Yang, N.; Tang, H. *5G NR and Enhancements: From R15 to R16*; Elsevier: Amsterdam, The Netherlands, 2022.
28. Jiang, J. *Key Technologies of C-V2X Internet of Vehicles and Vehicle-Road Coordination*; China Science and Technology Publishing & Media Co., Ltd.: Beijing, China, 2024.
29. Zhu, L.; Zhang, Z.; Liu, L.; Feng, L.; Lin, P.; Zhang, Y. Online distributed learning-based load-aware heterogeneous vehicular edge computing. *IEEE Sens. J.* **2023**, *23*, 17350–17365. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.